

Estrutura Lineares

Aula 03 – Listas, Filas e Pilhas



C.E.S.A.R.

Tipos de Dados Abstratos (ADT)

Tipos de Dados Abstratos (ADT)

ADT é um tipo de dados que implementa objetos cujo comportamento é definido por um conjunto de valores e operações.

- O conceito da estrutura é separado de sua implementação subjacente.
- O que importa é **o que** ela faz, e não **como** ela faz.
- Empregadas para simplificar diversas operações em programação.

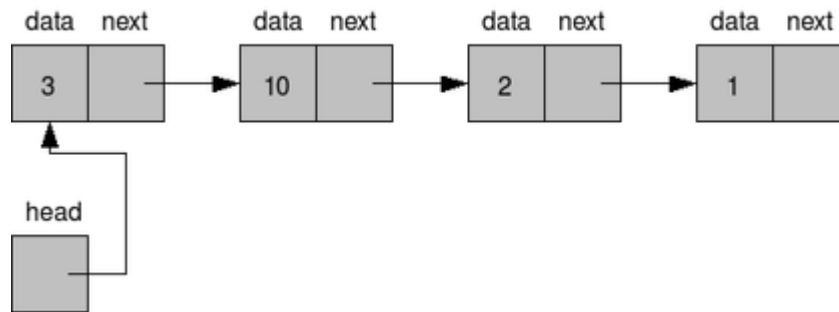
Os principais tipos de ADTs são:

- Lista: permite inserção e remoção em qualquer posição.
- Pilha (Stack): segue a regra LIFO (Last In, First Out).
- Fila (Queue): segue a regra FIFO (First In, First Out).

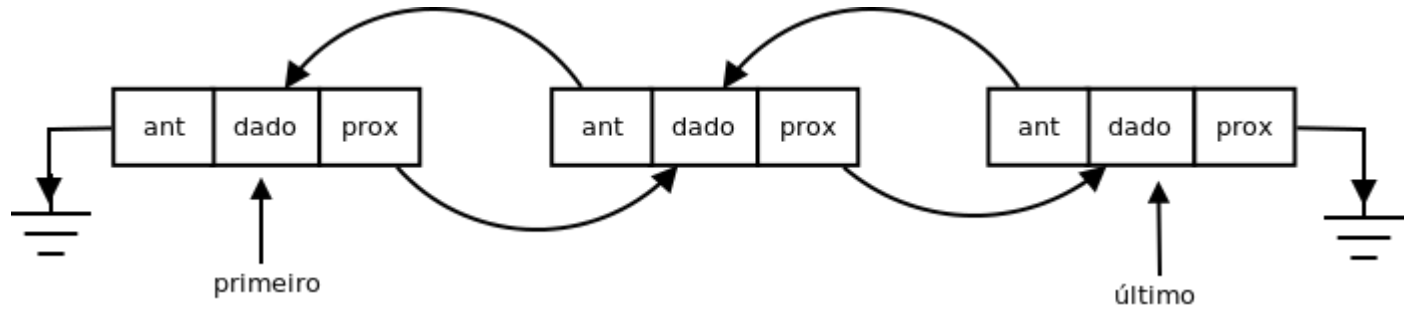
Listas

Listas (Linked List)

- Em uma lista os dados são armazenados de forma sequencial, em estruturas chamadas de nós.
- O principal tipo de lista é chamado de lista encadeada (linked list).
- Os dados são armazenados em nós, estruturas individuais compostas por um campo de informação e um (ponteiro de ligação - link), que conecta ao próximo nó na lista.
- A lista pode ser simples ou dupla. Uma lista duplamente encadeada contém também ponteiros para elementos anteriores.
- Uma lista encadeada também pode ser do tipo Circular.
- Amplamente empregadas na implementação de pilhas e filas.



Listas Duplamente Encadeada



Listas Vantagens

- A inserção ou remoção de um elemento na lista não implica a mudança de lugar de outros elementos;
- Não é necessário definir, no momento da criação da lista, o número máximo de elementos que esta poderá ter. Ou seja, é possível alocar memória "dinamicamente", apenas para o número de nós necessários.

Listas Desvantagens

- A manipulação torna-se mais "perigosa" uma vez que, se o encadeamento (ligação) entre elementos da lista for mal feito, toda a lista pode ser perdida;
- Para procurar ao elemento na posição n da lista, deve-se percorrer todos os elementos $(n - 1)$ anteriores.

Operações em Listas

- Diversas operações podem ser realizadas em listas, tais como:
- `get()`: Retornar um elemento da lista, em uma posição específica
- `insert()`: Inserir um elemento em uma posição qualquer na lista
- `remove()`: Remover a primeira ocorrência de um elemento qualquer em uma lista (não-vazia)
- `removeAt()`: Remover o elemento qualquer em uma posição especificada na lista (não-vazia)
- `replace()`: Substituir um elemento em qualquer posição por outro elemento
- `size()`: Retornar o tamanho da lista (número de elementos)
- `isEmpty()`: retorna um booleano informando se a estrutura está vazia
- `isFull()`: retorna um booleano informando se a estrutura está cheia

Operações em Listas

```
typedef struct No {  
    int dado;  
    struct No* prox;  
} No;
```

```
No* inserir_inicio(No* head, int valor){  
    No* novo = (No*)malloc(sizeof(No));  
    novo->dado = valor;  
    novo->prox = head;  
    return novo;  
}
```

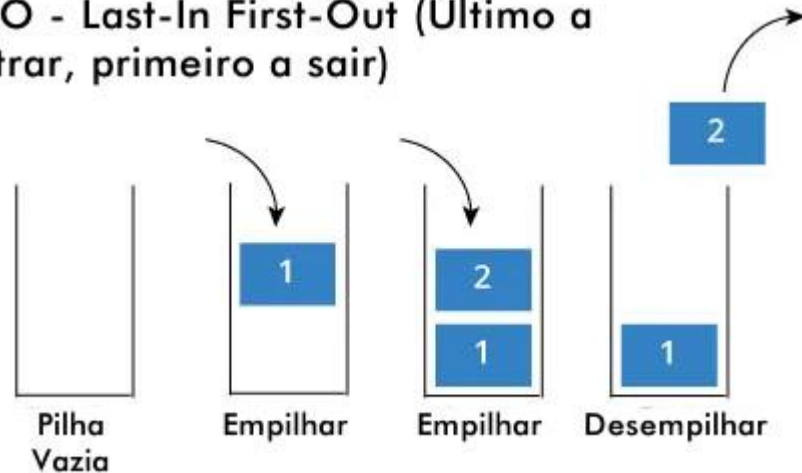
```
void imprimir_lista(No* head){  
    No* atual = head;  
    while (atual != NULL){  
        cout << atual->dado << ".\n";  
        atual = atual->prox;  
    }  
    cout << "NULL \n";  
}
```

Pilha

Pilha

- Estrutura linear para armazenamento de itens que são inseridos e removidos de acordo com o princípio LIFO (last-in-first-out)
- Os objetos podem ser inseridos a qualquer momento, mas apenas o objeto que foi inserido por último pode ser removido.
- Os elementos podem ser inseridos e excluídos apenas em um lado da lista, chamado de topo.

LIFO - Last-In First-Out (Último a entrar, primeiro a sair)



Operações em Pilha

Diversas operações podem ser realizadas em pilhas, tais como:

- `push()`: Inserir um item o topo da pilha
- `pop()`: Remover e retornar o elemento do topo da pilha, se não estiver vazia.
- `peek()`: Retorna o elemento no topo da pilha, sem removê-lo.
- `size()`: retorna o tamanho da pilha (no de itens)
- `isEmpty()`: retorna um booleano informando se a pilha está vazia
- `isFull()`: retorna um booleano informando se a pilha está cheia



Aplicações das Pilhas

As pilhas encontram inúmeras aplicações em desenvolvimento de algoritmos e software em geral:

- Implementação de funções recursivas
- Mecanismos de desfazer /refazer em aplicações gerais
- Verificação sintática em compiladores
- Avaliação de expressões aritméticas em certas calculadoras
- Operações de backtracking (em rotas de mapas e jogos, por exemplo)
- Avaliação de expressões em geral
- NLP (Natural Language Processing)

Operações com Pilhas

```
typedef struct Pilha {  
    int dado;  
    struct Pilha* prox;  
} Pilha;
```

Push (inserir elemento na pilha)

```
Pilha* push(Pilha* topo, int valor){  
    Pilha* novo = (Pilha*) malloc(sizeof(Pilha));  
    novo->dado = valor;  
    novo->prox = topo;  
    return novo;  
}
```

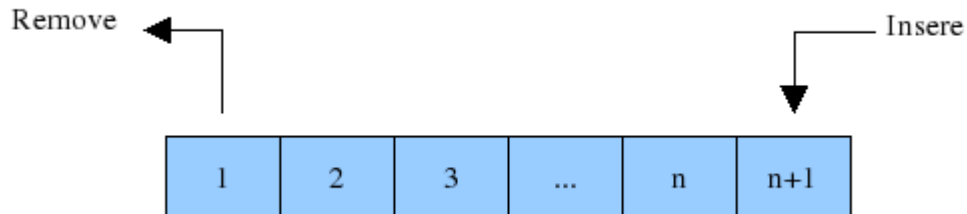
Pop (remover elemento do topo)

```
Pilha* pop(Pilha* topo){  
    if (topo == NULL)  
        return NULL;  
    Pilha* temp = topo;  
    topo = topo->prox;  
    free(temp);  
    return topo;  
}
```

Fila

Fila

- Estrutura linear na qual a inserção e remoção de itens segue o princípio FIFO (first-in-first-out)
- Os objetos podem ser inseridos a qualquer momento, mas apenas o objeto que está há mais tempo na fila pode ser removido.
- Elementos somente são inseridos em um lado da lista, o final, e somente podem ser excluídos do outro lado, o início da lista.
- Cada nó contém um ponteiro para os dados e um ponteiro de ligação (link) para o próximo elemento na fila.



Operações em Filas

- Diversas operações podem ser realizadas em filas, tais como:
- enqueue(): Insere um item no final da fila
- dequeue(): Retira e retorna o elemento que está no início da fila, se ela não estiver vazia.
- peek(): Retorna um elemento da fila, sem removê-lo, se ela não estiver vazia.
- size(): retorna o tamanho da pilha (no de itens)
- isEmpty(): retorna um booleano informando se a pilha está vazia
- isFull(): retorna um booleano informando se a pilha está cheia



Operações em Filas

São usadas como ferramentas para criação de algoritmos, assim como as pilhas.

Exemplos de uso de filas incluem:

- Resposta a requisições de serviços compartilhados, como filas de impressão e acesso a disco
- Armazenamento de teclas digitadas no teclado
- Agendamento de tempo de CPU
- Transferência de dados assíncrona entre processos (buffers, E/S em arquivos, etc.)
- Contadores de tíquetes e gerenciamento de filas por senhas (em hospitais, por exemplo) e listas de espera em geral

Operações em Filas

```
typedef struct Fila {  
    int dado;  
  
    struct Fila* prox;  
} Fila;
```

```
Fila* enqueue(Fila* fim, int valor){  
    Fila* novo = (Fila*)malloc(sizeof(Fila));  
    novo->dado = valor;  
    novo->prox = NULL;  
    if (fim != NULL) fim->prox = novo;  
    return novo;  
}
```

```
Fila* dequeue(Fila* inicio){  
    if (inicio == NULL) return NULL;  
    Fila* temp = inicio;  
    inicio = inicio->prox;  
    free(temp);  
    return inicio;  
}
```

Pilha x Fila

Pilha x Fila

Pilha	Fila
Baseadas LIFO: o último elemento a ser inserido é o primeiro a ser retirado	Baseadas em FIFO: o primeiro elemento a ser inserido é também o primeiro a ser retirado
Inserção e exclusão somente ocorrem em uma ponta da lista, o topo	Inserção e exclusão ocorrem nos dois lados da lista: inserção no final e exclusão no início.
Operação de inserção: push() Operação de exclusão: pop()	Operação de inserção: enqueue() Operação de exclusão: dequeue()
Um ponteiro é necessário para acessar a lista, que sempre aponta para o topo.	Dois ponteiros são necessários: um aponta para o primeiro elemento e o outro, para o último
Usada quando precisamos acessar o elemento mais recentemente adicionado.	Usada quando precisamos acessar os elementos em ordem de chegada, ou prioridade

Exercícios Listas

Implemente uma lista encadeada simples em C/C++ com as seguintes funcionalidades:

- Inserir elementos no início da lista.
- Inserir elementos no final da lista.
- Imprimir todos os elementos da lista.

Ex:

Inserir no início: 10, 20, 30

Inserir no final: 40, 50

Saída esperada: 30 -> 20 -> 10 -> 40 -> 50 -> NULL

Exercícios Pilha

Inverter uma Palavra com Pilha

Crie um programa que receba uma palavra e a inverta utilizando uma pilha.

Entrada Exemplo: CADERNO

Saída: ONREDAC

Exercícios Filas

Simulador de Atendimento (Fila)

Crie um programa que simule uma fila de atendimento. O programa deve permitir:

Adicionar uma pessoa na fila.

Atender a próxima pessoa (removê-la da fila).

Exibir a fila atual.

Exemplo: Adicionar: Ana, Bruno, Carlos

Fila atual: Ana <- Bruno <- Carlos <- NULL

Atender: Ana

Fila atual: Bruno <- Carlos <- NULL



C . E . S . A . R

Pessoas impulsionando inovação.
Inovação impulsionando negócios.

